

QairtTensor

Updated: Jul 02, 2026

80-63442-10

Rev: AL

Provide Feedback

ⓘ **Note**

Some methods in this module are not yet implemented in the current release and will raise an exception if called. See the C API for full functionality.

Include: `#include "QairtCppApi/QairtTensor.hpp"`

C++ wrapper for the QAIRT tensor API.

Provides `data-type`, quantization-`encoding`, and tensor classes used to describe and transfer tensor `data` across QAIRT backends.

namespace qairt

Enums

enum class DataType : std::underlying_type_t<QairtDataType_t>

Element data types supported for tensor data.

Enumerator	Description
Int2	Signed 2-bit integer.

Int4	Signed 4-bit integer.
Int8	Signed 8-bit integer.
Int16	Signed 16-bit integer.
Int32	Signed 32-bit integer.
Int64	Signed 64-bit integer.
UInt2	Unsigned 2-bit integer.
UInt4	Unsigned 4-bit integer.
UInt8	Unsigned 8-bit integer.
UInt16	Unsigned 16-bit integer.
UInt32	Unsigned 32-bit integer.
UInt64	Unsigned 64-bit integer.
Float4	4-bit floating-point.
Float8	8-bit floating-point.
Float16	16-bit floating-point (IEEE 754 half precision).
BFloat16	16-bit brain floating-point (bfloat16).
Float32	32-bit floating-point (IEEE 754 single).
Float64	64-bit floating-point (IEEE 754 double).
SFixedPoint2	Signed 2-bit fixed-point.

QAIRT Datatype	Description
SFixedPoint4	Signed 4-bit fixed-point.
SFixedPoint8	Signed 8-bit fixed-point.
SFixedPoint16	Signed 16-bit fixed-point.
SFixedPoint32	Signed 32-bit fixed-point.
UFixedPoint2	Unsigned 2-bit fixed-point.
UFixedPoint4	Unsigned 4-bit fixed-point.
UFixedPoint8	Unsigned 8-bit fixed-point.
UFixedPoint16	Unsigned 16-bit fixed-point.
UFixedPoint32	Unsigned 32-bit fixed-point.
Bool8	8-bit boolean.
String	Variable-length string.
Undefined	Unspecified or unknown data type.

Values:

enumerator Int2 = [QAIRT_DATATYPE_INT_2](#)

enumerator Int4 = [QAIRT_DATATYPE_INT_4](#)

enumerator Int8 = [QAIRT_DATATYPE_INT_8](#)

enumerator Int16 = [QAIRT_DATATYPE_INT_16](#)



enumerator Int32 = QAIRT_DATATYPE_INT_32

enumerator Int64 = QAIRT_DATATYPE_INT_64

enumerator UInt2 = QAIRT_DATATYPE_UINT_2

enumerator UInt4 = QAIRT_DATATYPE_UINT_4

enumerator UInt8 = QAIRT_DATATYPE_UINT_8

enumerator UInt16 = QAIRT_DATATYPE_UINT_16

enumerator UInt32 = QAIRT_DATATYPE_UINT_32

enumerator UInt64 = QAIRT_DATATYPE_UINT_64

enumerator Float4 = QAIRT_DATATYPE_FLOAT_4

enumerator Float8 = QAIRT_DATATYPE_FLOAT_8

enumerator Float16 = QAIRT_DATATYPE_FLOAT_16

enumerator BFloat16 = QAIRT_DATATYPE_BFLOAT_16

enumerator Float32 = QAIRT_DATATYPE_FLOAT_32

enumerator Float64 = QAIRT_DATATYPE_FLOAT_64

enumerator SFixedPoint2 = QAIRT_DATATYPE_SFIXED_POINT_2

enumerator SFixedPoint4 = QAIRT_DATATYPE_SFIXED_POINT_4

Provide Feedback



enumerator SFixedPoint8 = QAIRT_DATATYPE_SFIXED_POINT_8

enumerator SFixedPoint16 = QAIRT_DATATYPE_SFIXED_POINT_16

enumerator SFixedPoint32 = QAIRT_DATATYPE_SFIXED_POINT_32

enumerator UFixedPoint2 = QAIRT_DATATYPE_UFIXED_POINT_2

enumerator UFixedPoint4 = QAIRT_DATATYPE_UFIXED_POINT_4

enumerator UFixedPoint8 = QAIRT_DATATYPE_UFIXED_POINT_8

enumerator UFixedPoint16 = QAIRT_DATATYPE_UFIXED_POINT_16

enumerator UFixedPoint32 = QAIRT_DATATYPE_UFIXED_POINT_32

enumerator Bool8 = QAIRT_DATATYPE_BOOL_8

enumerator String = QAIRT_DATATYPE_STRING

enumerator Undefined = QAIRT_DATATYPE_UNDEFINED

enum class QuantizationEncoding :
std::underlying_type_t<Qairt_QuantizationEncoding_t>

Quantization encoding types for tensor data.

Enumerator	Description
ScaleOffset	Per-tensor scale-offset encoding.
AxisScaleOffset	Per-axis (e.g., per-channel) scale-offset encoding.

Provide Feedback



<code>BwScaleOffset</code>	Bit-width scale-offset encoding.
<code>BwAxisScaleOffset</code>	Bit-width per-axis scale-offset encoding.
<code>Block</code>	Per-block scale-offset encoding.
<code>BlockwiseExpansion</code>	Blockwise expansion encoding.
<code>Vector</code>	Vector quantization (VQ) compression encoding.
<code>BwAxisScaleOffsetMapped</code>	Bit-width per-axis scale-offset encoding with mapping.
<code>BwBlockMapped</code>	Bit-width per-block scale-offset encoding with mapping.
<code>BwBlockwiseExpansionMapped</code>	Bit-width blockwise expansion encoding with mapping.
<code>FloatBlock</code>	Per-block float scale-offset encoding.
<code>BwFloatBlock</code>	Bit-width per-block float scale-offset encoding.
<code>Microscaling</code>	Microscaling (MX) encoding.
<code>Undefined</code>	Unused sentinel; present to ensure a 32-bit enum storage.

Values:

```
enumerator ScaleOffset = QAIRT\_QUANTIZATION\_ENCODING\_SCALE\_OFFSET
```

```
enumerator AxisScaleOffset =  
QAIRT\_QUANTIZATION\_ENCODING\_AXIS\_SCALE\_OFFSET
```

```
enumerator BwScaleOffset = QAIRT\_QUANTIZATION\_ENCODING\_BW\_SCALE\_OFFSET
```

enumerator BwAxisScaleOffset =

QAIRT_QUANTIZATION_ENCODING_BW_AXIS_SCALE_OFFSET

enumerator Block = QAIRT_QUANTIZATION_ENCODING_BLOCK

enumerator BlockwiseExpansion =

QAIRT_QUANTIZATION_ENCODING_BLOCKWISE_EXPANSION

enumerator Vector = QAIRT_QUANTIZATION_ENCODING_VECTOR

enumerator BwAxisScaleOffsetMapped =

QAIRT_QUANTIZATION_ENCODING_BW_AXIS_SCALE_OFFSET_MAPPED

enumerator BwBlockMapped =

QAIRT_QUANTIZATION_ENCODING_BW_BLOCK_MAPPED

enumerator BwBlockwiseExpansionMapped =

QAIRT_QUANTIZATION_ENCODING_BW_BLOCKWISE_EXPANSION_MAPPED

enumerator FloatBlock = QAIRT_QUANTIZATION_ENCODING_FLOAT_BLOCK

enumerator BwFloatBlock = QAIRT_QUANTIZATION_ENCODING_BW_FLOAT_BLOCK

enumerator Microscaling = QAIRT_QUANTIZATION_ENCODING_MICROSCALING

enumerator Undefined = QAIRT_QUANTIZATION_ENCODING_UNDEFINED

enum class QuantizationEncodingMapping :

std::underlying_type_t<Qairt_QuantizationEncodingMapping_t>

Quantized value mapping schemes for scale-offset encodings.

Provide Feedback

Enumerator	Description
StandardSymmetric	Standard symmetric two's complement mapping.
AsymmetricPlusOne	Two's complement mapping with a positive shift of one.
LinearSymmetricExcludeZero	Linear mapping symmetric about zero, excluding zero from the range.
Undefined	Unused sentinel; present to ensure a 32-bit enum storage.

Values:

```
enumerator StandardSymmetric =
QAIRT_QUANTIZATION_ENCODING_MAPPING_STANDARD_SYMMETRIC
```

```
enumerator AsymmetricPlusOne =
QAIRT_QUANTIZATION_ENCODING_MAPPING_ASYMMETRIC_PLUS_ONE
```

```
enumerator LinearSymmetricExcludeZero =
QAIRT_QUANTIZATION_ENCODING_MAPPING_LINEAR_SYMMETRIC_EXCLUDE_ZERO
```

```
enumerator Undefined = QAIRT_QUANTIZATION_ENCODING_MAPPING_UNDEFINED
```

```
enum class FloatEncoding : std::underlying_type_t<Qairt_FloatEncoding_t>
```

Floating-point sub-format encodings used in [Microscaling](#) (MX) quantization.

Enumerator	Description
MXFP8_E5M2	MXFP8 format with 5 exponent bits and 2 mantissa bits; compatible with Float8.
MXFP8_E4M3	MXFP8 format with 4 exponent bits and 3 mantissa bits; compatible with Float8.
MXFP6_E3M2	MXFP6 format with 3 exponent bits and 2 mantissa bits.

Provide Feedback



MXFP6_E2M3	MXFP6 format with 2 exponent bits and 3 mantissa bits.
MXFP4_E2M1	MXFP4 format with 2 exponent bits and 1 mantissa bit.
Undefined	Unused sentinel; present to ensure a 32-bit enum storage.

Values:

enumerator MXFP8_E5M2 = QAIRT_FLOAT_ENCODING_MXFP8_E5M2

enumerator MXFP8_E4M3 = QAIRT_FLOAT_ENCODING_MXFP8_E4M3

enumerator MXFP6_E3M2 = QAIRT_FLOAT_ENCODING_MXFP6_E3M2

enumerator MXFP6_E2M3 = QAIRT_FLOAT_ENCODING_MXFP6_E2M3

enumerator MXFP4_E2M1 = QAIRT_FLOAT_ENCODING_MXFP4_E2M1

enumerator Undefined = QAIRT_FLOAT_ENCODING_UNDEFINED

enum class TensorMemType : std::underlying_type_t<QairtTensor_MemoryType_t>

Memory access strategies for tensor data.

Enumerator	Description
Raw	Raw memory pointer provided directly by the client.
MemHandle	Shared memory object handle enabling memory sharing across backends.
RetrieveRaw	Callback-based retrieval; the backend calls client-supplied callbacks to fetch raw data.

Provide Feedback



Undefined	Unused sentinel; present to ensure a 32-bit enum storage.

Values:

enumerator Raw =

enumerator MemHandle =

enumerator RetrieveRaw =

enumerator Undefined =

enum class BlockwiseExpansionBlockScaleStorageType :
std::underlying_type_t<

Provide Feedback

Storage bit-widths for block scales in blockwise expansion quantization.

Enumerator	Description
Storage8	Block scales stored as 8-bit values.
Storage16	Block scales stored as 16-bit values.
Undefined	Unused sentinel; present to ensure a 32-bit enum storage.

Values:

enumerator Storage8 =

enumerator Storage16 =



enum class QairtQuantizeParamsEncoding -

QAIRT_BLOCKWISE_EXPANSION_BITWIDTH_SCALE_STORAGE_UNDEFINED

```
class AxisScaleOffset : public qairt::ApiType<AxisScaleOffset,  
QairtQuantizeParams_AxisScaleOffsetV1_t>
```

```
#include <QairtTensor.hpp>
```

Per-axis (e.g., per-channel) collection of scale-offset pairs for axis scale-offset quantization.

Construct directly: `AxisScaleOffset()`. Set the quantization axis via `setAxis()` and supply one `ScaleOffset` per axis element via `setScaleOffsets()`. Attach to a `QuantizeParams` via `QuantizeParams::setAxisScaleOffsetEncoding()`.

Public Functions

```
inline ~AxisScaleOffset()
```

```
AxisScaleOffset() noexcept = default
```

```
inline AxisScaleOffset(AxisScaleOffset &&other) noexcept
```

```
inline AxisScaleOffset &operator=(AxisScaleOffset &&other) noexcept
```

```
inline AxisScaleOffset(const AxisScaleOffset &other)
```

```
inline AxisScaleOffset &operator=(const AxisScaleOffset &other)
```

```
inline AxisScaleOffset shallowCopy() const
```



```
inline int32_t getAxis() const
```

See also

[QairtQuantizeParams_AxisScaleOffset_getAxis](#)

```
inline void setAxis(int32_t axis) const
```

See also

[QairtQuantizeParams_AxisScaleOffset_setAxis](#)

Provide Feedback

```
inline std::vector<ScaleOffset> &getScaleOffsets()
```

Get the per-axis scale-offset pairs for this encoding.

Throws

[qairt::Exception](#) – on invalid handle.

Returns

Reference to the vector of [ScaleOffset](#) objects, one per axis element.

```
inline const std::vector<ScaleOffset> &getScaleOffsets() const
```

Wrapper which allows for const access to the per-axis scale-offset pairs.

See also

[AxisScaleOffset::getScaleOffsets\(\)](#)

```
inline void setScaleOffsets(const std::vector<ScaleOffset> &scaleOffsets)
```

Set the per-axis scale-offset pairs for this encoding.

Parameters



Throws

`qairt::Exception` – on invalid handle.

```
template<typename T, typename U, typename V>  
inline ApiType(const ApiType<T, U, V> &parent,  
detail::non_owning_handle<handle_type> noh)
```

```
inline explicit ApiType(const std::shared_ptr<T_Table> &apiTable)
```

```
inline ApiType(copy_table_tag_t, const ApiType &other)
```

```
ApiType() noexcept = default
```

```
ApiType(const ApiType&) = delete
```

```
ApiType(ApiType&&) noexcept = default
```

```
ApiType &operator=(const ApiType&) = delete
```

```
ApiType &operator=(ApiType&&) noexcept = default
```

Provide Feedback

```
inline void prepareToCross() const
```

```
inline void updateAfterCross() const
```

```
inline explicit AxisScaleOffset(const std::shared_ptr<ApiTable> &apiTable)
```

Private Members

```
detail::crossable<std::vector<detail::non_owning<ScaleOffset>>,  
&interface_type::getScaleOffsetAt, &interface_type::getNumScaleOffsets,  
&interface_type::setScaleOffsets> m_scaleOffsets
```

Per-axis scale-offset pairs indexed along the quantization axis.

Friends

```
friend class Api
```

```
class BlockEncoding : public qairt::ApiType<BlockEncoding,  
QairtQuantizeParams_BlockEncodingV1_t>
```

```
#include <QairtTensor.hpp>
```

Block sizes and per-block scale-offset pairs for per-block scale-offset quantization.

Construct directly: `BlockEncoding(const std::vector<uint32_t>& blockSize,const std::vector<ScaleOffset>& scaleOffsets)`. Attach to a `QuantizeParams` via `QuantizeParams::setBlockEncoding()`.

Public Functions

```
inline ~BlockEncoding()
```

```
BlockEncoding() noexcept = default
```

```
inline BlockEncoding(BlockEncoding &&other) noexcept
```

```
inline BlockEncoding &operator=(BlockEncoding &&other) noexcept
```

```
inline BlockEncoding(const BlockEncoding &other)
```

```
inline BlockEncoding &operator=(const BlockEncoding &other)
```

```
inline BlockEncoding shallowCopy() const
```

```
inline BlockEncoding(const std::vector<uint32_t> &blockSize, const  
std::vector<ScaleOffset> &scaleOffsets)
```

Construct a block encoding with the given block sizes and per-block scale-offset pairs.

Parameters

- **blockSize** – [in] Block sizes, one per quantization dimension.
- **scaleOffsets** – [in] Scale-offset pairs, one per quantization block.

```
inline std::vector<uint32_t> getBlockSizes() const
```

Get the block sizes along each quantization dimension.

Returns

```
inline void setBlockSizes(const std::vector<uint32_t> &blockSizes) const
```

Set the block sizes along each quantization dimension.

Parameters

blockSizes – [in] Block sizes, one per quantization dimension.

```
inline void setBlockSizes(std::vector<uint32_t> &&blockSizes) const
```

See also

`BlockEncoding::setBlockSizes(const std::vector<uint32_t>&)`

```
inline std::vector<ScaleOffset> &getScaleOffsets()
```

Get the per-block scale-offset pairs for this encoding.

Throws

`qairt::Exception` – on invalid handle.

Returns

Reference to the vector of `ScaleOffset` objects, one per quantization block.

```
inline const std::vector<ScaleOffset> &getScaleOffsets() const
```

Wrapper which allows for const access to the per-block scale-offset pairs.

See also

`BlockEncoding::getScaleOffsets()`

```
inline void setScaleOffsets(std::vector<ScaleOffset> scaleOffsets)
```



Parameters

`scaleOffsets` – [in] Scale-offset pairs, one per quantization block.

Throws

`qairt::Exception` – on invalid handle.

```
template<typename T, typename U, typename V>  
inline ApiType(const ApiType<T, U, V> &parent,  
detail::non_owning_handle<handle_type> noh)
```

Provide Feedback

```
inline explicit ApiType(const std::shared_ptr<T_Table> &apiTable)
```

```
inline ApiType(copy_table_tag_t, const ApiType &other)
```

```
ApiType() noexcept = default
```

```
ApiType(const ApiType&) = delete
```

```
ApiType(ApiType&&) noexcept = default
```

```
ApiType &operator=(const ApiType&) = delete
```

```
ApiType &operator=(ApiType&&) noexcept = default
```



Private Functions

```
inline void prepareToCross() const
```

```
inline void updateAfterCross() const
```

```
inline explicit BlockEncoding(const std::shared_ptr<ApiTable> &apiTable)
```

Provide Feedback

Private Members

```
friend Api
```

```
mutable std::vector<uint32_t> m_blockSize
```

Block sizes along each quantization dimension.

```
detail::crossable<std::vector<detail::non_owning<ScaleOffset>>,  
&interface_type::getScaleOffsetAt, &interface_type::getNumScaleOffsets,  
&interface_type::setScaleOffsets> m_scaleOffsets
```

Scale-offset pairs for each quantization block.

```
class BlockwiseExpansion : public qairt::ApiType<BlockwiseExpansion,  
QairtQuantizeParams_BlockwiseExpansionV1_t>
```

```
#include <QairtTensor.hpp>
```

Per-axis blockwise expansion quantization parameters, including axis, per-block scale-offset pairs, and block scale data.

Construct directly: BlockwiseExpansion(int32_t axis,

const std::vector<ScaleOffset>& scaleOffsets, uint32_t numBlocksPerAxis,

uint32_t blockScaleBitwidth). Supply block scale data as either 8-bit values via setBlocksScale8() or 16-bit values via setBlocksScale16() — these are mutually exclusive.



[QuantizeParams](#) via `QuantizeParams::setBlockwiseExpansion()`.

 Note

Setting 8-bit block scales clears any 16-bit block scales and vice versa.

Public Functions

```
inline ~BlockwiseExpansion()
```

```
BlockwiseExpansion() noexcept = default
```

```
inline BlockwiseExpansion(BlockwiseExpansion &&other) noexcept
```

```
inline BlockwiseExpansion &operator=(BlockwiseExpansion &&other) noexcept
```

```
inline BlockwiseExpansion(const BlockwiseExpansion &other)
```

```
inline BlockwiseExpansion &operator=(const BlockwiseExpansion &other)
```

```
inline BlockwiseExpansion shallowCopy() const
```

```
inline BlockwiseExpansion(int32_t axis, const std::vector<ScaleOffset>  
&ScaleOffsets, uint32_t numBlocksPerAxis, uint32_t blockScaleBitwidth)
```

Provide Feedback



```
inline int32_t getAxis() const
```

```
inline void setAxis(int32_t axis) const
```

```
inline uint32_t getNumBlocksPerAxis() const
```

```
inline void setNumBlocksPerAxis(uint32_t setNumBlocks) const
```

```
inline uint32_t getBlockScaleBitwidth() const
```

```
inline void setBlockScaleBitwidth(uint32_t setBlockScaleBw) const
```

```
inline std::vector<ScaleOffset> &getScaleOffsets()
```

```
inline const std::vector<ScaleOffset> &getScaleOffsets() const
```

```
inline void setScaleOffsets(const std::vector<ScaleOffset> &scaleOffsets)
```

```
inline BlockwiseExpansionBlockScaleStorageType getStorageType() const
```

```
inline const std::vector<uint8_t> &getBlocksScale8() const
```

Provide Feedback



```
inline const std::vector<uint16_t> &getBlocksScale16() const
```

```
inline void setBlocksScale8(const std::vector<uint8_t> &blocksScale8)
```

```
inline void setBlocksScale16(const std::vector<uint16_t> &blocksScale16)
```

```
template<typename T, typename U, typename V>  
inline ApiType(const ApiType<T, U, V> &parent,  
detail::non_owning_handle<handle_type> noh)
```

```
inline explicit ApiType(const std::shared_ptr<T_Table> &apiTable)
```

```
inline ApiType(copy_table_tag_t, const ApiType &other)
```

```
ApiType() noexcept = default
```

```
ApiType(const ApiType&) = delete
```

```
ApiType(ApiType&&) noexcept = default
```

```
ApiType &operator=(const ApiType&) = delete
```

Provide Feedback



```
ApiType &operator=(ApiType&&) noexcept = default
```

Private Functions

```
inline void prepareToCross() const
```

```
inline void updateAfterCross() const
```

```
inline explicit BlockwiseExpansion(const std::shared_ptr<ApiTable> &apiTable)
```

Provide Feedback

Private Members

```
friend Api
```

```
detail::crossable<std::vector<detail::non_owning<ScaleOffset>>,  
&interface_type::getScaleOffsetAt, &interface_type::getNumScaleOffsets,  
&interface_type::setScaleOffsets> m_scaleOffsets
```

Scale-offset pairs for each quantization block in the expansion encoding.

```
mutable std::vector<uint8_t> m_blocksScale8
```

Block scale values stored as 8-bit data; mutually exclusive with m_blocksScale16.

```
mutable std::vector<uint16_t> m_blocksScale16
```

Block scale values stored as 16-bit data; mutually exclusive with m_blocksScale8.

```
class BwAxisScaleOffset : public qairt::ApiType<BwAxisScaleOffset,  
QairtQuantizeParams_BwAxisScaleOffsetV1_t>
```

```
#include <QairtTensor.hpp>
```



quantization.

Construct directly: `BwAxisScaleOffset(uint32_t bitwidth, int32_t axis, const std::vector<float>& scales, const std::vector<int32_t>& offsets)`. Attach to a `QuantizeParams` via `QuantizeParams::setBwAxisScaleOffsetEncoding()`, or supply as the codebook descriptor for `VectorEncoding`.

Public Functions

```
inline ~BwAxisScaleOffset()
```

```
BwAxisScaleOffset() noexcept = default
```

```
inline BwAxisScaleOffset(BwAxisScaleOffset &&other) noexcept
```

```
inline BwAxisScaleOffset &operator=(BwAxisScaleOffset &&other) noexcept
```

```
inline BwAxisScaleOffset(const BwAxisScaleOffset &other)
```

```
inline BwAxisScaleOffset &operator=(const BwAxisScaleOffset &other)
```

```
inline BwAxisScaleOffset shallowCopy() const
```

```
inline BwAxisScaleOffset(uint32_t bitwidth, int32_t axis, const std::vector<float>& scales, const std::vector<int32_t> &offsets)
```



Construct a bit-width per-axis scale-offset encoding with the given parameters.

- **bitwidth** – [in] Storage bit-width of the quantized values.
- **axis** – [in] Quantization axis index.
- **scales** – [in] Per-axis scale values, one per element along the quantization axis.
- **offsets** – [in] Per-axis integer zero-point offsets, one per axis element.

inline uint32_t getBitwidth() const

See also

[QairtQuantizeParams_BwAxisScaleOffset_getBw](#)

Provide Feedback

inline void setBitwidth(uint32_t bitwidth) const

See also

[QairtQuantizeParams_BwAxisScaleOffset_setBw](#)

inline int32_t getAxis() const

See also

[QairtQuantizeParams_BwAxisScaleOffset_getAxis](#)

inline void setAxis(int32_t axis) const

See also

[QairtQuantizeParams_BwAxisScaleOffset_setAxis](#)

inline const std::vector<float> &getScales() const

Get the per-axis scale values.



Reference to the vector of scale values, one per element along the quantization axis.

```
inline void setScales(const std::vector<float> &scales)
```

Set the per-axis scale values.

Parameters

scales – [in] Scale values, one per element along the quantization axis.

```
inline void setScales(std::vector<float> &&scales)
```

See also

[BwAxisScaleOffset::setScales\(const std::vector<float> &\)](#)

```
inline const std::vector<int32_t> &getOffsets() const
```

Get the per-axis integer offset values.

Returns

Reference to the vector of offset values, one per element along the quantization axis.

```
inline void setOffsets(const std::vector<int32_t> &offsets)
```

Set the per-axis integer offset values.

Parameters

offsets – [in] Integer zero-point offsets, one per element along the quantization axis.

```
inline void setOffsets(std::vector<int32_t> &&offsets)
```



```
BwAxisScaleOffset::setOffsets(const std::vector<int32_t>&)
```

```
template<typename T, typename U, typename V>  
inline ApiType(const ApiType<T, U, V> &parent,  
detail::non_owning_handle<handle_type> noh)
```

```
inline explicit ApiType(const std::shared_ptr<T_Table> &apiTable)
```

```
inline ApiType(copy_table_tag_t, const ApiType &other)
```

```
ApiType() noexcept = default
```

```
ApiType(const ApiType&) = delete
```

```
ApiType(ApiType&&) noexcept = default
```

```
ApiType &operator=(const ApiType&) = delete
```

```
ApiType &operator=(ApiType&&) noexcept = default
```

Private Functions

```
inline void prepareToCross() const
```

Provide Feedback



```
inline void updateAfterCross() const
```

```
inline explicit BwAxisScaleOffset(const std::shared_ptr<ApiTable> &apiTable)
```

Private Members

```
friend Api
```

```
mutable std::vector<float> m_scales
```

Per-axis scale values, one per element along the quantization axis.

```
mutable std::vector<int32_t> m_offsets
```

Per-axis offset values, one per element along the quantization axis.

```
class BwAxisScaleOffsetMapped : public qairt::ApiType<BwAxisScaleOffsetMapped,  
QairtQuantizeParams_BwAxisScaleOffsetMappedV1_t>
```

```
#include <QairtTensor.hpp>
```

Bit-width, per-axis scales and offsets, and a quantization mapping for bit-width per-axis scale-offset mapped quantization.

Construct directly: BwAxisScaleOffsetMapped(). Supply scales via [setScales\(\)](#), offsets via [setOffsets\(\)](#), and a mapping scheme via [setMapping\(\)](#). Attach to a [QuantizeParams](#) via [QuantizeParams::setBwAxisScaleOffsetMappedEncoding\(\)](#).

Public Functions

```
inline ~BwAxisScaleOffsetMapped()
```

```
BwAxisScaleOffsetMapped() noexcept = default
```

Provide Feedback



```
inline BwAxisScaleOffsetMapped(BwAxisScaleOffsetMapped &&other) noexcept
```

```
inline BwAxisScaleOffsetMapped &operator=(BwAxisScaleOffsetMapped &&other)  
noexcept
```

```
inline BwAxisScaleOffsetMapped(const BwAxisScaleOffsetMapped &other)
```

```
inline BwAxisScaleOffsetMapped &operator=(const BwAxisScaleOffsetMapped  
&other)
```

```
inline BwAxisScaleOffsetMapped shallowCopy() const
```

```
inline uint32_t getBitwidth() const
```

See also

[QairtQuantizeParams_BwAxisScaleOffsetMapped_getBw](#)

```
inline void setBitwidth(uint32_t bitwidth) const
```

See also

[QairtQuantizeParams_BwAxisScaleOffsetMapped_setBw](#)

```
inline int32_t getAxis() const
```

See also

```
inline void setAxis(int32_t axis) const
```

See also

[QairtQuantizeParams_BwAxisScaleOffsetMapped_setAxis](#)

```
inline QuantizationEncodingMapping getMapping() const
```

See also

[QairtQuantizeParams_BwAxisScaleOffsetMapped_getMapping](#)

Provide Feedback

```
inline void setMapping(QuantizationEncodingMapping mapping) const
```

See also

[QairtQuantizeParams_BwAxisScaleOffsetMapped_setMapping](#)

```
inline const std::vector<float> &getScales() const
```

Get the per-axis scale values for this encoding.

Returns

Reference to the vector of scale values, one per element along the quantization axis.

```
inline void setScales(const std::vector<float> &scales)
```

Set the per-axis scale values for this encoding.

Parameters

scales – [in] Scale values, one per element along the quantization axis.



```
inline void setScales(std::vector<float> &&scales)
```

See also

[BwAxisScaleOffsetMapped::setScales\(const std::vector<float>&\)](#)

```
inline const std::vector<int32_t> &getOffsets() const
```

Get the per-axis offset values for this encoding.

Returns

Reference to the vector of offset values, one per element along the quantization axis

Provide Feedback

```
inline void setOffsets(const std::vector<int32_t> &offsets)
```

Set the per-axis offset values for this encoding.

Parameters

offsets – [in] Integer zero-point offsets, one per element along the quantization axis.

```
inline void setOffsets(std::vector<int32_t> &&offsets)
```

See also

[BwAxisScaleOffsetMapped::setOffsets\(const std::vector<int32_t>&\)](#)

```
template<typename T, typename U, typename V>  
inline ApiType(const ApiType<T, U, V> &parent,  
detail::non_owing_handle<handle_type> noh)
```

```
inline explicit ApiType(const std::shared_ptr<T_Table> &apiTable)
```



```
inline ApiType(copy_table_tag_t, const ApiType &other)
```

```
ApiType() noexcept = default
```

```
ApiType(const ApiType&) = delete
```

```
ApiType(ApiType&&) noexcept = default
```

```
ApiType &operator=(const ApiType&) = delete
```

```
ApiType &operator=(ApiType&&) noexcept = default
```

Provide Feedback

Private Functions

```
inline void prepareToCross() const
```

```
inline void updateAfterCross() const
```

```
inline explicit BwAxisScaleOffsetMapped(const std::shared_ptr<ApiTable>  
&apiTable)
```

Private Members

```
friend Api
```



```
mutable std::vector<float> m_scales
```

Per-axis scale values for the mapped encoding, one per quantization axis element.

```
mutable std::vector<int32_t> m_offsets
```

Per-axis offset values for the mapped encoding, one per quantization axis element.

```
class BwBlockMapped : public qairt::ApiType<BwBlockMapped,  
QairtQuantizeParams\_BwBlockMappedV1\_t>
```

```
#include <QairtTensor.hpp>
```

Bit-width, block sizes, a quantization mapping, and per-block scale-offset pairs for bit-width per-block mapped quantization.

Construct directly: `BwBlockMapped()`. Supply block sizes via [setBlockSizes\(\)](#), a mapping scheme via [setMapping\(\)](#), and per-block [ScaleOffset](#) values via [setScaleOffsets\(\)](#). Attach to a [QuantizeParams](#) via `QuantizeParams::setBwBlockMapped()`.

Public Functions

```
inline ~BwBlockMapped()
```

```
BwBlockMapped() noexcept = default
```

```
inline BwBlockMapped(BwBlockMapped &&other) noexcept
```

```
inline BwBlockMapped &operator=(BwBlockMapped &&other) noexcept
```

```
inline BwBlockMapped(const BwBlockMapped &other)
```

```
inline BwBlockMapped &operator=(const BwBlockMapped &other)
```

```
inline BwBlockMapped shallowCopy() const
```

```
inline uint32_t getBitwidth() const
```

See also

[QairtQuantizeParams_BwBlockMapped_getBw](#)

```
inline void setBitwidth(uint32_t bitwidth) const
```

See also

[QairtQuantizeParams_BwBlockMapped_setBw](#)

```
inline QuantizationEncodingMapping getMapping() const
```

See also

[QairtQuantizeParams_BwBlockMapped_getMapping](#)

```
inline void setMapping(QuantizationEncodingMapping mapping) const
```

See also

[QairtQuantizeParams_BwBlockMapped_setMapping](#)

```
inline const std::vector<uint32_t> &getBlockSizes() const
```

Get the block sizes along each quantization dimension.

Returns

Provide Feedback



```
inline void setBlockSizes(const std::vector<uint32_t> &blockSizes)
```

Set the block sizes along each quantization dimension.

Parameters

blockSizes – [in] Block sizes, one per quantization dimension.

```
inline void setBlockSizes(std::vector<uint32_t> &&blockSizes)
```

See also

[BwBlockMapped::setBlockSizes\(const std::vector<uint32_t>&\)](#)

```
inline void setBlockSize(std::vector<uint32_t> blockSizes)
```

See also

[BwBlockMapped::setBlockSizes\(const std::vector<uint32_t>&\)](#)

```
inline std::vector<ScaleOffset> &getScaleOffsets()
```

Get the per-block scale-offset pairs for this encoding.

Throws

[qairt::Exception](#) – on invalid handle.

Returns

Reference to the vector of [ScaleOffset](#) objects, one per quantization block.

```
inline const std::vector<ScaleOffset> &getScaleOffsets() const
```

Wrapper which allows for const access to the per-block scale-offset pairs.

```
BwBlockMapped::getScaleOffsets()
```

```
inline void setScaleOffsets(std::vector<ScaleOffset> scaleOffsets)
```

Set the per-block scale-offset pairs for this encoding.

Parameters

`scaleOffsets` – [in] Scale-offset pairs, one per quantization block.

Throws

`qairt::Exception` – on invalid handle.

Provide Feedback

```
template<typename T, typename U, typename V>
inline ApiType(const ApiType<T, U, V> &parent,
detail::non_owing_handle<handle_type> noh)
```

```
inline explicit ApiType(const std::shared_ptr<T_Table> &apiTable)
```

```
inline ApiType(copy_table_tag_t, const ApiType &other)
```

```
ApiType() noexcept = default
```

```
ApiType(const ApiType&) = delete
```

```
ApiType(ApiType&&) noexcept = default
```



```
ApiType &operator=(const ApiType&) = delete
```

```
ApiType &operator=(ApiType&&) noexcept = default
```

Private Functions

```
inline void prepareToCross() const
```

```
inline void updateAfterCross() const
```

```
inline explicit BwBlockMapped(const std::shared_ptr<ApiTable> &apiTable)
```

Provide Feedback

Private Members

```
friend Api
```

```
mutable std::vector<uint32_t> m_blockSizes
```

Block sizes along each quantization dimension.

```
detail::crossable<std::vector<detail::non_owning<ScaleOffset>>,  
&interface_type::getScaleOffsetAt, &interface_type::getNumScaleOffsets,  
&interface_type::setScaleOffsets> m_scaleOffsets
```

Scale-offset pairs for each quantization block.

```
class BwBlockwiseExpansionMapped : public  
qairt::ApiType<BwBlockwiseExpansionMapped,  
QairtQuantizeParams_BwBlockwiseExpansionMappedV1_t>
```

```
#include <QairtTensor.hpp>
```



data for bit-width blockwise expansion mapped quantization.

Construct directly: `BwBlockwiseExpansionMapped()`. Supply block scale data as either 8-bit values via `setBlocksScale8()` or 16-bit values via `setBlocksScale16()` — these are mutually exclusive. Attach to a `QuantizeParams` via `QuantizeParams::setBwBlockwiseExpansionMapped()`.

Note

Setting 8-bit block scales clears any 16-bit block scales and vice versa.

Public Functions

```
inline ~BwBlockwiseExpansionMapped()
```

```
BwBlockwiseExpansionMapped() noexcept = default
```

```
inline BwBlockwiseExpansionMapped(BwBlockwiseExpansionMapped &&other)  
noexcept
```

```
inline BwBlockwiseExpansionMapped &operator=(BwBlockwiseExpansionMapped  
&&other) noexcept
```

```
inline BwBlockwiseExpansionMapped(const BwBlockwiseExpansionMapped &other)
```

```
inline BwBlockwiseExpansionMapped &operator=(const  
BwBlockwiseExpansionMapped &other)
```

```
inline BwBlockwiseExpansionMapped shallowCopy() const
```

```
inline uint32_t getBitwidth() const
```

See also

[QairtQuantizeParams_BwBlockwiseExpansionMapped_getBw](#)

```
inline void setBitwidth(uint32_t bitwidth) const
```

See also

[QairtQuantizeParams_BwBlockwiseExpansionMapped_setBw](#)

```
inline QuantizationEncodingMapping getMapping() const
```

See also

[QairtQuantizeParams_BwBlockwiseExpansionMapped_getMapping](#)

```
inline void setMapping(QuantizationEncodingMapping mapping) const
```

See also

[QairtQuantizeParams_BwBlockwiseExpansionMapped_setMapping](#)

```
inline int32_t getAxis() const
```

See also

[QairtQuantizeParams_BwBlockwiseExpansionMapped_getAxis](#)

```
inline void setAxis(int32_t axis) const
```



[QairtQuantizeParams_BwBlockwiseExpansionMapped_setAxis](#)

```
inline uint32_t getNumBlocksPerAxis() const
```

See also

[QairtQuantizeParams_BwBlockwiseExpansionMapped_getNumBlocksPerAxis](#)

```
inline void setNumBlocksPerAxis(uint32_t numBlocksPerAxis) const
```

See also

[QairtQuantizeParams_BwBlockwiseExpansionMapped_setNumBlocksPerAxis](#)

```
inline uint32_t getBlockScaleBitwidth() const
```

See also

[QairtQuantizeParams_BwBlockwiseExpansionMapped_getBlockScaleBitwidth](#)

```
inline void setBlockScaleBitwidth(uint32_t blockScaleBitwidth) const
```

See also

[QairtQuantizeParams_BwBlockwiseExpansionMapped_setBlockScaleBitwidth](#)

```
inline std::vector<ScaleOffset> &getScaleOffsets()
```

Get the per-block scale-offset pairs for this encoding.

Throws

[qairt::Exception](#) – on invalid handle.

Returns

Provide Feedback



```
inline const std::vector<ScaleOffset> &getScaleOffsets() const
```

Wrapper which allows for const access to the per-block scale-offset pairs.

See also

[BwBlockwiseExpansionMapped::getScaleOffsets\(\)](#)

```
inline void setScaleOffsets(const std::vector<ScaleOffset> &scaleOffsets)
```

Set the per-block scale-offset pairs for this encoding.

Parameters

scaleOffsets – [in] Scale-offset pairs, one per quantization block.

Throws

[qairt::Exception](#) – on invalid handle.

```
inline const std::vector<uint8_t> &getBlocksScale8() const
```

Get the 8-bit block scale values.

Returns

Reference to the vector of 8-bit scale values.

Note

Valid only when block scales were set via [setBlocksScale8\(\)](#).

```
inline const std::vector<uint16_t> &getBlocksScale16() const
```

Get the 16-bit block scale values.

Reference to the vector of 16-bit scale values.

Note

Valid only when block scales were set via `setBlocksScale16()`.

```
inline void setBlocksScale8(const std::vector<uint8_t> &blocksScale8)
```

Set the 8-bit block scale values.

Clears any previously set 16-bit block scales.

Parameters

`blocksScale8` – [in] 8-bit block scale values, one per quantization block.

```
inline void setBlocksScale16(const std::vector<uint16_t> &blocksScale16)
```

Set the 16-bit block scale values.

Clears any previously set 8-bit block scales.

Parameters

`blocksScale16` – [in] 16-bit block scale values, one per quantization block.

```
template<typename T, typename U, typename V>  
inline ApiType(const ApiType<T, U, V> &parent,  
detail::non_owning_handle<handle_type> noh)
```

```
inline explicit ApiType(const std::shared_ptr<T_Table> &apiTable)
```

```
inline ApiType(copy_table_tag_t, const ApiType &other)
```

```
ApiType() noexcept = default
```

```
ApiType(const ApiType&) = delete
```

```
ApiType(ApiType&&) noexcept = default
```

```
ApiType &operator=(const ApiType&) = delete
```

```
ApiType &operator=(ApiType&&) noexcept = default
```

Private Functions

```
inline void prepareToCross() const
```

```
inline void updateAfterCross() const
```

```
inline explicit BwBlockwiseExpansionMapped(const std::shared_ptr<ApiTable>  
&apiTable)
```

Private Members

```
friend Api
```

Provide Feedback



```
detail::crossable<std::vector<detail::non_owning<ScaleOffset>,
&interface_type::getScaleOffsetAt, &interface_type::getNumScaleOffsets,
&interface_type::setScaleOffsets> m_scaleOffsets
```

Scale-offset pairs for each quantization block in the mapped encoding.

```
mutable std::vector<uint8_t> m_blocksScale8
```

Block scale values stored as 8-bit data; mutually exclusive with m_blocksScale16.

```
mutable std::vector<uint16_t> m_blocksScale16
```

Block scale values stored as 16-bit data; mutually exclusive with m_blocksScale8.

```
class BwFloatBlockEncoding : public qairt::ApiType<BwFloatBlockEncoding,
QairtQuantizeParams_BwFloatBlockEncodingV1_t>
```

```
#include <QairtTensor.hpp>
```

Bit-width and per-block floating-point scale-offset pairs for bit-width float block quantization.

Construct directly: BwFloatBlockEncoding(). Supply block sizes via [setBlockSizes\(\)](#) and one [FloatScaleOffset](#) per block via [setFloatScaleOffsets\(\)](#). Attach to a [QuantizeParams](#) via [QuantizeParams::setBwFloatBlockEncoding\(\)](#).

Public Functions

```
inline ~BwFloatBlockEncoding()
```

```
BwFloatBlockEncoding() noexcept = default
```

```
inline BwFloatBlockEncoding(BwFloatBlockEncoding &&other) noexcept
```

```
inline BwFloatBlockEncoding &operator=(BwFloatBlockEncoding &&other) noexcept
```



```
inline BwFloatBlockEncoding(const BwFloatBlockEncoding &other)
```

```
inline BwFloatBlockEncoding &operator=(const BwFloatBlockEncoding &other)
```

```
inline BwFloatBlockEncoding shallowCopy() const
```

```
inline uint32_t getBitwidth() const
```

See also

[QairtQuantizeParams_BwFloatBlockEncoding_getBw](#)

```
inline void setBitwidth(uint32_t bitwidth) const
```

See also

[QairtQuantizeParams_BwFloatBlockEncoding_setBw](#)

```
inline const std::vector<uint32_t> &getBlockSizes() const
```

Get the block sizes along each quantization dimension.

Returns

Reference to the vector of block sizes.

```
inline void setBlockSizes(const std::vector<uint32_t> &blockSizes)
```

Set the block sizes along each quantization dimension.

Parameters

```
inline void setBlockSizes(std::vector<uint32_t> &&blockSizes)
```

See also

[BwFloatBlockEncoding::setBlockSizes\(const std::vector<uint32_t>&\)](#)

```
inline std::vector<FloatScaleOffset> &getFloatScaleOffsets()
```

Get the per-block floating-point scale-offset pairs.

Throws

[qairt::Exception](#) – on invalid handle.

Returns

Reference to the vector of [FloatScaleOffset](#) objects, one per quantization block.

```
inline const std::vector<FloatScaleOffset> &getFloatScaleOffsets() const
```

Wrapper which allows for const access to the per-block float scale-offset pairs.

See also

[BwFloatBlockEncoding::getFloatScaleOffsets\(\)](#)

```
inline void setFloatScaleOffsets(std::vector<FloatScaleOffset> floatScaleOffsets)
```

Set the per-block floating-point scale-offset pairs.

Parameters

floatScaleOffsets – [in] Float scale-offset pairs, one per quantization block.

Throws

[qairt::Exception](#) – on invalid handle.

```
template<typename T, typename U, typename V>
inline ApiType(const ApiType<T, U, V> &parent,
detail::non_owing_handle<handle_type> noh)
```

```
inline explicit ApiType(const std::shared_ptr<T_Table> &apiTable)
```

```
inline ApiType(copy_table_tag_t, const ApiType &other)
```

```
ApiType() noexcept = default
```

```
ApiType(const ApiType&) = delete
```

```
ApiType(ApiType&&) noexcept = default
```

```
ApiType &operator=(const ApiType&) = delete
```

```
ApiType &operator=(ApiType&&) noexcept = default
```

Private Functions

```
inline void prepareToCross() const
```

```
inline void updateAfterCross() const
```

Provide Feedback



```
inline explicit BwFloatBlockEncoding(const std::shared_ptr<ApiTable> &apiTable)
```

Private Members

```
friend Api
```

```
mutable std::vector<uint32_t> m_blockSizes
```

Block size along each quantization dimension for the float block encoding.

```
detail::crossable<std::vector<detail::non_owning<FloatScaleOffset>>,  
&interface_type::getFloatScaleOffsetAt, &interface_type::getNumFloatScaleOffsets,  
&interface_type::setFloatScaleOffsets> m_floatScaleOffsets
```

Float scale-offset pairs, one per quantization block.

```
class BwScaleOffset : public qairt::ApiType<BwScaleOffset,  
QairtQuantizeParams_BwScaleOffsetV1_t>
```

```
#include <QairtTensor.hpp>
```

Bit-width, scale, and integer offset for bit-width scale-offset quantization.

Construct directly: `BwScaleOffset(uint32_t bitwidth, float scale, int32_t offset)`. Attach to a `QuantizeParams` via `QuantizeParams::setBwScaleOffsetEncoding()`.

Public Functions

```
inline ~BwScaleOffset()
```

```
BwScaleOffset() noexcept = default
```

```
inline BwScaleOffset(BwScaleOffset &&other) noexcept
```



```
inline BwScaleOffset &operator=(BwScaleOffset &&other) noexcept
```

```
inline BwScaleOffset(const BwScaleOffset &other)
```

```
inline BwScaleOffset &operator=(const BwScaleOffset &other)
```

```
inline BwScaleOffset shallowCopy() const
```

```
inline BwScaleOffset(uint32_t bitwidth, float scale, int32_t offset)
```

Construct a bit-width scale-offset encoding with the given parameters.

Parameters

- **bitwidth** – [in] Storage bit-width of the quantized values.
- **scale** – [in] Scale factor.
- **offset** – [in] Integer zero-point offset.

```
inline uint32_t getBitwidth() const
```

See also

[QairtQuantizeParams_BwScaleOffset_getBw](#)

```
inline void setBitwidth(uint32_t bitwidth) const
```

See also

[QairtQuantizeParams_BwScaleOffset_setBw](#)

inline float getScale() const

See also

[QairtQuantizeParams_BwScaleOffset_getScale](#)

inline void setScale(float **scale) const**

See also

[QairtQuantizeParams_BwScaleOffset_setScale](#)

inline int32_t getOffset() const

See also

[QairtQuantizeParams_BwScaleOffset_getOffset](#)

inline void setOffset(int32_t **offset) const**

See also

[QairtQuantizeParams_BwScaleOffset_setOffset](#)

template<typename T, typename U, typename V>
inline ApiType(const [ApiType](#)<T, U, V> &parent**,**
[detail::non_owing_handle](#)<handle_type> **noh)**

inline explicit ApiType(const std::shared_ptr<T_Table> &apiTable**)**

inline ApiType(copy_table_tag_t, const [ApiType](#) &other**)**



```
ApiType() noexcept = default
```

```
ApiType(const ApiType&) = delete
```

```
ApiType(ApiType&&) noexcept = default
```

```
ApiType &operator=(const ApiType&) = delete
```

```
ApiType &operator=(ApiType&&) noexcept = default
```

Provide Feedback

Private Functions

```
inline void prepareToCross() const
```

```
inline void updateAfterCross() const
```

```
inline explicit BwScaleOffset(const std::shared_ptr<ApiTable> &apiTable)
```

Friends

```
friend class Api
```

```
class ClientBuffer : public qairt::ApiType<ClientBuffer, QairtTensor_ClientBufferV1_t>
```

```
#include <QairtTensor.hpp>
```



Construct directly: `ClientBuffer()`. Associate with a [Tensor](#) via [TensorMemory::setClientBuffer\(\)](#).

Public Functions

`ClientBuffer()` noexcept = default

`ClientBuffer(const ClientBuffer&) = delete`

`ClientBuffer &operator=(const ClientBuffer&) = delete`

`ClientBuffer(ClientBuffer&&) noexcept = default`

`ClientBuffer &operator=(ClientBuffer&&) noexcept = default`

`inline void *getData()`

See also

[QairTensor_ClientBuffer_getData](#)

`inline const void *getData() const`

See also

[QairTensor_ClientBuffer_getData](#)

`inline void setData(void *data)`



[QairtTensor_ClientBuffer_setData](#)

inline uint32_t getDataSize() const

See also

[QairtTensor_ClientBuffer_getDataSize](#)

inline void setDataSize(uint32_t dataSize)

See also

[QairtTensor_ClientBuffer_setDataSize](#)

Provide Feedback

Private Functions

inline explicit ClientBuffer(const std::shared_ptr<ApiTable> &apiTable)

inline void prepareToCross() const

inline void updateAfterCross() const

Private Members

friend Api

Friends

***friend class* TensorMemory**



```
class FloatBlockEncoding : public qairt::ApiType<FloatBlockEncoding,
QairtQuantizeParams_FloatBlockEncodingV1_t>
```

```
#include <QairtTensor.hpp>
```

Per-block floating-point scale-offset pairs for float block quantization.

Construct directly: FloatBlockEncoding(). Supply block sizes via [setBlockSizes\(\)](#) and one [FloatScaleOffset](#) per block via [setFloatScaleOffsets\(\)](#). Attach to a [QuantizeParams](#) via [QuantizeParams::setFloatBlockEncoding\(\)](#).

Public Functions

```
inline ~FloatBlockEncoding()
```

```
FloatBlockEncoding() noexcept = default
```

```
inline FloatBlockEncoding(FloatBlockEncoding &&other) noexcept
```

```
inline FloatBlockEncoding &operator=(FloatBlockEncoding &&other) noexcept
```

```
inline FloatBlockEncoding(const FloatBlockEncoding &other)
```

```
inline FloatBlockEncoding &operator=(const FloatBlockEncoding &other)
```

```
inline FloatBlockEncoding shallowCopy() const
```

```
inline const std::vector<uint32_t> &getBlockSizes() const
```

Get the block sizes along each quantization dimension.

Returns

Reference to the vector of block sizes.

```
inline void setBlockSizes(const std::vector<uint32_t> &blockSizes)
```

Set the block sizes along each quantization dimension.

Parameters

blockSizes – [in] Block sizes, one per quantization dimension.

Provide Feedback

```
inline void setBlockSizes(std::vector<uint32_t> &&blockSizes)
```

See also

[FloatBlockEncoding::setBlockSizes\(const std::vector<uint32_t>&\)](#)

```
inline std::vector<FloatScaleOffset> &getFloatScaleOffsets()
```

Get the per-block floating-point scale-offset pairs.

Throws

[qairt::Exception](#) – on invalid handle.

Returns

Reference to the vector of [FloatScaleOffset](#) objects, one per quantization block.

```
inline const std::vector<FloatScaleOffset> &getFloatScaleOffsets() const
```

Wrapper which allows for const access to the per-block float scale-offset pairs.



FloatBlockEncoding::getFloatScaleOffsets()

```
inline void setFloatScaleOffsets(std::vector<FloatScaleOffset> floatScaleOffsets)
```

Set the per-block floating-point scale-offset pairs.

Parameters

floatScaleOffsets – [in] Float scale-offset pairs, one per quantization block.

Throws

`qairt::Exception` – on invalid handle.

Provide Feedback

```
template<typename T, typename U, typename V>
inline ApiType(const ApiType<T, U, V> &parent,
detail::non_owning_handle<handle_type> noh)
```

```
inline explicit ApiType(const std::shared_ptr<T_Table> &apiTable)
```

```
inline ApiType(copy_table_tag_t, const ApiType &other)
```

```
ApiType() noexcept = default
```

```
ApiType(const ApiType&) = delete
```

```
ApiType(ApiType&&) noexcept = default
```



```
ApiType &operator=(const ApiType&) = delete
```

```
ApiType &operator=(ApiType&&) noexcept = default
```

Private Functions

```
inline void prepareToCross() const
```

```
inline void updateAfterCross() const
```

```
inline explicit FloatBlockEncoding(const std::shared_ptr<ApiTable> &apiTable)
```

Provide Feedback

Private Members

```
friend Api
```

```
mutable std::vector<uint32_t> m_blockSizes
```

Block size along each quantization dimension for the float block encoding.

```
detail::crossable<std::vector<detail::non_owning<FloatScaleOffset>>,  
&interface_type::getFloatScaleOffsetAt, &interface_type::getNumFloatScaleOffsets,  
&interface_type::setFloatScaleOffsets> m_floatScaleOffsets
```

Float scale-offset pairs, one per quantization block.

```
class FloatScaleOffset : public qairt::ApiType<FloatScaleOffset,  
QairtQuantizeParams_FloatScaleOffsetV1_t>
```

```
#include <QairtTensor.hpp>
```

Floating-point scale and offset pair for float block quantization encodings.



BwFloatBlockEncoding and FloatBlockEncoding.

Public Functions

```
inline ~FloatScaleOffset()
```

```
FloatScaleOffset() noexcept = default
```

```
inline FloatScaleOffset(FloatScaleOffset &&other) noexcept
```

```
inline FloatScaleOffset &operator=(FloatScaleOffset &&other) noexcept
```

```
inline FloatScaleOffset(const FloatScaleOffset &other)
```

```
inline FloatScaleOffset &operator=(const FloatScaleOffset &other)
```

```
inline FloatScaleOffset shallowCopy() const
```

```
inline FloatScaleOffset(float scale, float offset)
```

Construct a floating-point scale-offset pair with the given scale and offset.

Parameters

- **scale** – [in] Scale factor.
- **offset** – [in] Floating-point zero-point offset.

Provide Feedback



inline float getScale()

See also

[QairtQuantizeParams_FloatScaleOffset_getScale](#)

inline float getScale() const

See also

[QairtQuantizeParams_FloatScaleOffset_getScale](#)

inline void setScale(float **scale) const**

See also

[QairtQuantizeParams_FloatScaleOffset_setScale](#)

inline float getOffset()

See also

[QairtQuantizeParams_FloatScaleOffset_getOffset](#)

inline float getOffset() const

See also

[QairtQuantizeParams_FloatScaleOffset_getOffset](#)

inline void setOffset(float **offset) const**

See also

[QairtQuantizeParams_FloatScaleOffset_setOffset](#)

Provide Feedback



```
template<typename T, typename U, typename V>  
inline ApiType(const ApiType<T, U, V> &parent,  
detail::non_owning_handle<handle_type> noh)
```

```
inline explicit ApiType(const std::shared_ptr<T_Table> &apiTable)
```

```
inline ApiType(copy_table_tag_t, const ApiType &other)
```

```
ApiType() noexcept = default
```

```
ApiType(const ApiType&) = delete
```

```
ApiType(ApiType&&) noexcept = default
```

```
ApiType &operator=(const ApiType&) = delete
```

```
ApiType &operator=(ApiType&&) noexcept = default
```

Private Functions

```
inline void prepareToCross() const
```

```
inline void updateAfterCross() const
```

Provide Feedback



```
inline explicit FloatScaleOffset(const std::shared_ptr<ApiTable> &apiTable)
```

Private Members

```
friend Api
```

```
class Microscaling : public qairt::ApiType<Microscaling,  
QairtQuantizeParams_MicroscalingEncodingV1_t>
```

```
#include <QairtTensor.hpp>
```

Microscaling (MX) quantization parameters, including float encoding, block dimensions, and per-block scale values.

Construct directly: `Microscaling()`. Set the value encoding format via `setValueEncoding()`, block dimensions via `setBlockDimensions()`, and block scale data via `setBlockScales8()` (for Float8 scale type) or `setBlockScalesFloat()` (for Float16 or Float32 scale type). Attach to a `QuantizeParams` via `QuantizeParams::setMicroscalingEncoding()`.

Note

Setting 8-bit block scales implicitly sets the scale data type to Float8; setting float block scales implicitly sets it to Float32.

Public Functions

```
inline ~Microscaling()
```

```
Microscaling() noexcept = default
```

```
inline Microscaling(Microscaling &&other) noexcept
```

```
inline Microscaling &operator=(Microscaling &&other) noexcept
```

```
inline Microscaling(const Microscaling &other)
```

```
inline Microscaling &operator=(const Microscaling &other)
```

```
inline Microscaling shallowCopy() const
```

```
inline FloatEncoding getValueEncoding() const
```

See also

[QairtQuantizeParams_MicroscalingEncoding_getValueEncoding](#)

```
inline void setValueEncoding(FloatEncoding valueEncoding) const
```

See also

[QairtQuantizeParams_MicroscalingEncoding_setValueEncoding](#)

```
inline const std::vector<uint32_t> &getBlockDimensions() const
```

Get the block dimensions for this microscaling encoding.

Returns

Reference to the vector of block dimension sizes, one per quantization dimension.

```
inline void setBlockDimensions(const std::vector<uint32_t> &blockDimensions)
```



Parameters

blockDimensions – [in] Block sizes, one per quantization dimension.

```
inline void setBlockDimensions(std::vector<uint32_t> &&blockDimensions)
```

See also

[Microscaling::setBlockDimensions\(const std::vector<uint32_t>&\)](#)

```
inline size_t getBlockCount() const
```

Get the number of blocks in this microscaling encoding.

Returns

Number of block scale values currently stored, or 0 if none have been set.

```
inline void setScaleDataType(DataType dtype) const
```

Set the data type used to store block scale values.

Initializes the internal block-scale storage to match the given data type. Float8 uses a `std::vector<uint8_t>` buffer; Float16 and Float32 use a `std::vector<float>` buffer. Calling this does not clear existing scale data unless the storage type changes.

See also

[QairtQuantizeParams_MicroscalingEncoding_setScaleDataType](#)

Parameters

dtype – [in] Data type for block scales. Must be Float8, Float16, or Float32.

Throws

`qairt::Exception` – on invalid handle.

```
inline DataType getScaleDataType() const
```

Get the data type used to store block scale values.

See also

[QairtQuantizeParams_MicroscalingEncoding_getScaleDataType](#)

Throws

`qairt::Exception` – on invalid handle.

Returns

The currently set scale data type.

Provide Feedback

```
inline const std::vector<uint8_t> &getBlockScales8() const
```

Get the 8-bit block scale values.

Valid when the scale data type is Float8.

Returns

Reference to the vector of 8-bit scale values.

```
inline void setBlockScales8(const std::vector<uint8_t> &blockScales8)
```

Set the 8-bit block scale values.

Implicitly sets the scale data type to Float8. Replaces any previously set float block scales.

Parameters

`blockScales8` – [in] 8-bit block scale values, one per quantization block.

```
inline void setBlockScales8(std::vector<uint8_t> &&blockScales8)
```

See also



```
inline const std::vector<float> &getBlockScalesFloat() const
```

Get the floating-point block scale values.

Valid when the scale data type is Float16 or Float32.

Returns

Reference to the vector of float scale values.

```
inline void setBlockScalesFloat(const std::vector<float> &blockScalesFloat)
```

Set the floating-point block scale values.

Implicitly sets the scale data type to Float32. Replaces any previously set 8-bit block scales.

Parameters

blockScalesFloat – [in] Float block scale values, one per quantization block.

```
inline void setBlockScalesFloat(std::vector<float> &&blockScalesFloat)
```

See also

[Microscaling::setBlockScalesFloat\(const std::vector<float>&\)](#)

```
template<typename T, typename U, typename V>  
inline ApiType(const ApiType<T, U, V> &parent,  
detail::non_owning_handle<handle_type> noh)
```

```
inline explicit ApiType(const std::shared_ptr<T_Table> &apiTable)
```



```
inline ApiType(copy_table_tag_t, const ApiType &other)
```

```
ApiType() noexcept = default
```

```
ApiType(const ApiType&) = delete
```

```
ApiType(ApiType&&) noexcept = default
```

```
ApiType &operator=(const ApiType&) = delete
```

```
ApiType &operator=(ApiType&&) noexcept = default
```

Provide Feedback

Private Functions

```
inline void prepareToCross() const
```

```
inline void updateAfterCross() const
```

```
inline explicit Microscaling(const std::shared_ptr<ApiTable> &apiTable)
```

Private Members

```
friend Api
```



```
mutable std::vector<uint8_t> m_blockDimensions
```

Size of each quantization block along each dimension.

```
mutable std::variant<std::monostate, std::vector<uint8_t>, std::vector<float>>>  
m_blockScales
```

Block scale values; holds uint8_t for Float8 scale data type or float otherwise.

```
class QuantizeParams : public qairt::ApiType<QuantizeParams,  
QairtQuantizeParams_V1_t>
```

```
#include <QairtTensor.hpp>
```

Container for the active quantization encoding applied to a tensor.

Construct directly: QuantizeParams(). Set exactly one encoding variant by calling the corresponding setter (e.g., setScaleOffsetEncoding(), setBwAxisScaleOffsetEncoding()). Attach to a [Tensor](#) via Tensor::setQuantizeParams(). The active encoding type is identified at runtime via getQuantizationEncoding().

Public Functions

```
inline ~QuantizeParams()
```

```
QuantizeParams() noexcept = default
```

```
inline QuantizeParams(QuantizeParams &&other) noexcept
```

```
inline QuantizeParams &operator=(QuantizeParams &&other) noexcept
```

```
inline QuantizeParams(const QuantizeParams &other)
```

```
inline QuantizeParams shallowCopy() const
```

```
inline QuantizeParams &operator=(const QuantizeParams &other)
```

```
inline QuantizationEncoding getQuantizationEncoding() const
```

```
inline void setScaleOffsetEncoding(ScaleOffset scaleOffset)
```

```
inline const ScaleOffset &getScaleOffsetEncoding() const
```

```
inline ScaleOffset &getScaleOffsetEncoding()
```

```
inline void setAxisScaleOffsetEncoding(AxisScaleOffset axisScaleOffset)
```

```
inline AxisScaleOffset &getAxisScaleOffsetEncoding()
```

```
inline const AxisScaleOffset &getAxisScaleOffsetEncoding() const
```

```
inline void setBwScaleOffsetEncoding(BwScaleOffset bwScaleOffset)
```

```
inline BwScaleOffset &getBwScaleOffsetEncoding()
```

Provide Feedback



```
inline const BwScaleOffset &getBwScaleOffsetEncoding() const
```

```
inline void setBwAxisScaleOffsetMappedEncoding(BwAxisScaleOffsetMapped  
bwAxisScaleOffsetMapped)
```

```
inline BwAxisScaleOffsetMapped &getBwAxisScaleOffsetMappedEncoding()
```

```
inline const BwAxisScaleOffsetMapped &getBwAxisScaleOffsetMappedEncoding()  
const
```

```
inline void setMicroscalingEncoding(Microscaling microscaling)
```

```
inline Microscaling &getMicroscalingEncoding()
```

```
inline const Microscaling &getMicroscalingEncoding() const
```

```
inline void setBwAxisScaleOffsetEncoding(BwAxisScaleOffset bwAxisScaleOffset)
```

```
inline BwAxisScaleOffset &getBwAxisScaleOffsetEncoding()
```

```
inline const BwAxisScaleOffset &getBwAxisScaleOffsetEncoding() const
```



```
inline void setBlockEncoding(BlockEncoding blockEncoding)
```

```
inline BlockEncoding &getBlockEncoding()
```

```
inline const BlockEncoding &getBlockEncoding() const
```

```
inline void setVectorEncoding(VectorEncoding vectorEncoding)
```

```
inline VectorEncoding &getVectorEncoding()
```

```
inline const VectorEncoding &getVectorEncoding() const
```

```
inline void setBlockwiseExpansion(BlockwiseExpansion blockwiseExpansion)
```

```
inline BlockwiseExpansion &getBlockwiseExpansion()
```

```
inline const BlockwiseExpansion &getBlockwiseExpansion() const
```

```
inline void setBwFloatBlockEncoding(BwFloatBlockEncoding  
bwFloatBlockEncoding)
```

Provide Feedback

```
inline BwFloatBlockEncoding &getBwFloatBlockEncoding()
```

```
inline const BwFloatBlockEncoding &getBwFloatBlockEncoding() const
```

```
inline void setFloatBlockEncoding(FloatBlockEncoding floatBlockEncoding)
```

```
inline FloatBlockEncoding &getFloatBlockEncoding()
```

```
inline const FloatBlockEncoding &getFloatBlockEncoding() const
```

```
inline void setBwBlockMapped(BwBlockMapped bwBlockMapped)
```

```
inline const BwBlockMapped &getBwBlockMapped() const
```

```
inline void setBwBlockwiseExpansionMapped(BwBlockwiseExpansionMapped  
bwBlockwiseExpansionMapped)
```

```
inline BwBlockwiseExpansionMapped &getBwBlockwiseExpansionMapped()
```

```
inline const BwBlockwiseExpansionMapped &getBwBlockwiseExpansionMapped()  
const
```

Provide Feedback



```
template<typename T, typename U, typename V>  
inline ApiType(const ApiType<T, U, V> &parent,  
detail::non_ownership_handle<handle_type> noh)
```

```
inline explicit ApiType(const std::shared_ptr<T_Table> &apiTable)
```

```
inline ApiType(copy_table_tag_t, const ApiType &other)
```

```
ApiType() noexcept = default
```

```
ApiType(const ApiType&) = delete
```

```
ApiType(ApiType&&) noexcept = default
```

```
ApiType &operator=(const ApiType&) = delete
```

```
ApiType &operator=(ApiType&&) noexcept = default
```

Private Types

```
using CrossableScaleOffset = detail::crossable<detail::non_ownership<ScaleOffset>,  
&interface_type::getScaleOffset, &interface_type::setScaleOffset>
```

```
using CrossableAxisScaleOffset =  
detail::crossable<detail::non_owing<AxisScaleOffset>,  
&interface_type::getAxisScaleOffset, &interface_type::setAxisScaleOffset>
```

```
using CrossableBwScaleOffset =  
detail::crossable<detail::non_owing<BwScaleOffset>,  
&interface_type::getBwScaleOffset, &interface_type::setBwScaleOffset>
```

```
using CrossableBwAxisScaleOffsetMapped =  
detail::crossable<detail::non_owing<BwAxisScaleOffsetMapped>,  
&interface_type::getBwAxisScaleOffsetMapped,  
&interface_type::setBwAxisScaleOffsetMapped>
```

```
using CrossableMicroscaling = detail::crossable<detail::non_owing<Microscaling>,  
&interface_type::getMicroscalingEncoding,  
&interface_type::setMicroscalingEncoding>
```

```
using CrossableBwAxisScaleOffset =  
detail::crossable<detail::non_owing<BwAxisScaleOffset>,  
&interface_type::getBwAxisScaleOffset, &interface_type::setBwAxisScaleOffset>
```

```
using CrossableBlockEncoding =  
detail::crossable<detail::non_owing<BlockEncoding>,  
&interface_type::getBlockEncoding, &interface_type::setBlockEncoding>
```

```
using CrossableVectorEncoding =  
detail::crossable<detail::non_owing<VectorEncoding>,  
&interface_type::getVectorEncoding, &interface_type::setVectorEncoding>
```

```
using CrossableBlockwiseExpansion =  
detail::crossable<detail::non_owing<BlockwiseExpansion>,  
&interface_type::getBlockwiseExpansion, &interface_type::setBlockwiseExpansion>
```

```
using CrossableBwFloatBlockEncoding =  
detail::crossable<detail::non_owing<BwFloatBlockEncoding>,  
&interface_type::getBwFloatBlockEncoding,  
&interface_type::setBwFloatBlockEncoding>
```

```
using CrossableFloatBlockEncoding =  
detail::crossable<detail::non_owing<FloatBlockEncoding>,  
&interface_type::getFloatBlockEncoding, &interface_type::setFloatBlockEncoding>
```

```
using CrossableBwBlockMapped =  
detail::crossable<detail::non_owing<BwBlockMapped>,  
&interface_type::getBwBlockMapped, &interface_type::setBwBlockMapped>
```

```
using CrossableBwBlockwiseExpansionMapped =  
detail::crossable<detail::non_owing<BwBlockwiseExpansionMapped>,  
&interface_type::getBwBlockwiseExpansionMapped,  
&interface_type::setBwBlockwiseExpansionMapped>
```

Private Functions

```
inline explicit QuantizeParams(const std::shared_ptr<ApiTable> &apiTable)
```

```
inline void prepareToCross() const
```



```
inline void updateAfterCross() const
```

Private Members

```
mutable std::variant<std::monostate, CrossableScaleOffset, CrossableAxisScaleOffset,  
CrossableBwScaleOffset, CrossableBwAxisScaleOffsetMapped, CrossableMicroscaling,  
CrossableBwAxisScaleOffset, CrossableBlockEncoding, CrossableVectorEncoding,  
CrossableBlockwiseExpansion, CrossableBwFloatBlockEncoding,  
CrossableFloatBlockEncoding, CrossableBwBlockMapped,  
CrossableBwBlockwiseExpansionMapped> m_encoding
```

Active quantization encoding; the variant alternative indicates the encoding type.

Friends

```
friend class Api
```

```
class ScaleOffset : public qairt::ApiType<ScaleOffset,  
QairtQuantizeParams_ScaleOffsetV1_t>
```

```
#include <QairtTensor.hpp>
```

Per-tensor scale and integer offset for scale-offset quantization.

Construct directly: `ScaleOffset(float scale, int32_t offset)`. Attach to a `QuantizeParams` via `QuantizeParams::setScaleOffsetEncoding()`.

Public Functions

```
inline ~ScaleOffset()
```

```
ScaleOffset() noexcept = default
```

```
inline ScaleOffset(ScaleOffset &&other) noexcept
```



```
inline ScaleOffset &operator=(ScaleOffset &&other) noexcept
```

```
inline ScaleOffset(float scale, int32_t offset)
```

Construct a scale-offset pair with the given scale and offset.

Parameters

- **scale** – [in] Scale factor.
- **offset** – [in] Integer zero-point offset.

Provide Feedback

```
inline ScaleOffset(const ScaleOffset &other)
```

```
inline ScaleOffset &operator=(const ScaleOffset &other)
```

```
inline ScaleOffset shallowCopy() const
```

```
inline float getScale() const
```

See also

[QairtQuantizeParams_ScaleOffset_getScale](#)

```
inline void setScale(float scale) const
```

See also

[QairtQuantizeParams_ScaleOffset_setScale](#)



```
inline int32_t getOffset() const
```

See also

[QairtQuantizeParams_ScaleOffset_getOffset](#)

```
inline void setOffset(int32_t offset) const
```

See also

[QairtQuantizeParams_ScaleOffset_setOffset](#)

```
template<typename T, typename U, typename V>  
inline ApiType(const ApiType<T, U, V> &parent,  
detail::non_owning_handle<handle_type> noh)
```

```
inline explicit ApiType(const std::shared_ptr<T_Table> &apiTable)
```

```
inline ApiType(copy_table_tag_t, const ApiType &other)
```

```
ApiType() noexcept = default
```

```
ApiType(const ApiType&) = delete
```

```
ApiType(ApiType&&) noexcept = default
```

Provide Feedback

```
ApiType &operator=(const ApiType&) = delete
```

```
ApiType &operator=(ApiType&&) noexcept = default
```

Private Functions

```
inline void prepareToCross() const
```

```
inline void updateAfterCross() const
```

```
inline explicit ScaleOffset(const std::shared_ptr<ApiTable> &apiTable)
```

Provide Feedback

Friends

```
friend class Api
```

```
class Tensor : public qairt::ApiType<Tensor, QairtTensor_V1_t>
```

```
#include <QairtTensor.hpp>
```

Descriptor for a single named tensor, including its shape, data type, memory binding, and quantization parameters.

Obtained via [Graph](#) operations that return input or output tensors, or constructed directly: `Tensor()`. Set the shape via `setDimensions()`, the data type via `setDataType()`, a memory binding via `setTensorMemory()` or `setClientBuffer()`, and quantization parameters via `setQuantizeParams()`.

Public Functions

```
inline ~Tensor()
```



Tensor() = default

inline Tensor(const Tensor &other)

inline Tensor(Tensor &&other) noexcept

inline Tensor &operator=(Tensor &&other) noexcept

inline Tensor &operator=(const Tensor &other)

inline void setName(std::string str)

inline const std::string &getName() const

inline void setTensorProperties(const TensorProperties &tensorProperties)

inline TensorProperties &getTensorProperties()

inline const TensorProperties &getTensorProperties() const

inline void setDataFormat(uint32_t dataFormat)

Provide Feedback



```
inline uint32_t getDataFormat() const
```

```
inline void setDataType(DataType dataType)
```

```
inline DataType getDataType() const
```

```
inline QuantizeParams &getQuantizeParams()
```

```
inline const QuantizeParams &getQuantizeParams() const
```

```
inline void setQuantizeParams(const QuantizeParams &quantizeParams)
```

```
inline uint32_t getRank() const
```

```
inline void setDimensions(const std::vector<uint32_t> &dims)
```

```
inline void setDimensions(std::vector<uint32_t> &&dims)
```

```
inline const std::vector<uint32_t> &getDimensions() const
```

Provide Feedback



```
inline std::vector<uint32_t> &getDimensions()
```

```
inline TensorMemory &getTensorMemory()
```

```
inline const TensorMemory &getTensorMemory() const
```

```
inline void setTensorMemory(const TensorMemory &tensorMemory)
```

```
inline void setIsDynamicDimensions(const std::vector<bool> &isDynamicDims)
```

```
inline void setIsDynamicDimensions(const std::vector<detail::bool_wrapper>  
&isDynamicDims)
```

```
inline const std::vector<detail::bool_wrapper> &getIsDynamicDimensions() const
```

```
inline std::vector<detail::bool_wrapper> &getIsDynamicDimensions()
```

```
inline ClientBuffer &getClientBuffer()
```

```
inline const ClientBuffer &getClientBuffer() const
```

Provide Feedback



```
inline void setClientBuffer(const ClientBuffer &clientBuffer)
```

```
inline void setClientBuffer(RawBuffer &&buffer)
```

```
inline void setId(uint64_t id)
```

```
inline uint64_t getId() const
```

```
inline bool getIsProduced() const
```

```
inline Tensor shallowCopy() const
```

```
template<typename T, typename U, typename V>  
inline ApiType(const ApiType<T, U, V> &parent,  
detail::non_owning_handle<handle_type> noh)
```

```
inline explicit ApiType(const std::shared_ptr<T_Table> &apiTable)
```

```
inline ApiType(copy_table_tag_t, const ApiType &other)
```

```
ApiType() noexcept = default
```

Provide Feedback



```
ApiType(const ApiType&) = delete
```

```
ApiType(ApiType&&) noexcept = default
```

```
ApiType &operator=(const ApiType&) = delete
```

```
ApiType &operator=(ApiType&&) noexcept = default
```

Provide Feedback

Private Functions

```
inline Tensor(copy_table_tag_t, const Tensor &other)
```

```
inline explicit Tensor(const std::shared_ptr<ApiTable> &apiTable)
```

```
inline void prepareToCross() const
```

```
inline void updateAfterCross() const
```

Private Members

```
friend Api
```

```
detail::crossable<std::string, &interface_type::getName, &interface_type::setName>  
m_name
```



```
detail::crossable<detail::non_owing<TensorProperties>,&interface_type::getTensorProperties, &interface_type::setTensorProperties>  
m_properties
```

[Tensor](#) attribute flags (input, output, static, optional, etc.).

```
detail::crossable<detail::non_owing<TensorMemory>,&interface_type::getTensorMemory, &interface_type::setTensorMemory> m_memory
```

Memory descriptor specifying how tensor data is stored or retrieved.

```
detail::crossable<detail::non_owing<QuantizeParams>,&interface_type::getQuantizeParams, &interface_type::setQuantizeParams>  
m_quantParams
```

Quantization parameters describing the encoding for this tensor's data.

```
mutable std::vector<uint32_t> m_dims
```

Shape of this tensor as a list of dimension sizes, ordered from outermost to innermost.

```
mutable std::vector<detail::bool_wrapper> m_isDynamicDims
```

Per-dimension dynamic flags; true indicates that dimension is dynamic at runtime.

Friends

```
friend class ::qairt::ApiType
```

```
class TensorMemory : public qairt::ApiType<TensorMemory, QairtTensor_MemoryV1_t>
```

```
#include <QairtTensor.hpp>
```

Memory descriptor specifying how tensor data is stored or retrieved at runtime.

Construct directly: `TensorMemory()`. Attach to a [Tensor](#) via `Tensor::setTensorMemory()`. The memory type (Raw, MemHandle, or RetrieveRaw) determines which accessor fields are active.

Public Functions



```
TensorMemory() noexcept = default
```

```
TensorMemory(const TensorMemory&) = delete
```

```
TensorMemory(TensorMemory&&) noexcept = default
```

```
TensorMemory &operator=(const TensorMemory&) = delete
```

```
TensorMemory &operator=(TensorMemory&&) noexcept = default
```

```
inline TensorMemType getMemoryType() const
```

See also

[QairtTensor_Memory_getMemoryType](#)

```
inline ClientBuffer &getClientBuffer()
```

Get the client buffer associated with this tensor memory.

Valid only when the memory type is Raw. The returned reference is bound to this [TensorMemory](#) and is invalidated if modification operations are performed on the same component API.

See also

[QairtTensor_Memory_getClientBuffer](#)

Throws

[qairt::Exception](#) – on invalid handle or memory type mismatch.

Returns

```
inline const ClientBuffer &getClientBuffer() const
```

Wrapper which allows for const access to the associated client buffer.

See also

[TensorMemory::getClientBuffer\(\)](#)

```
inline void setClientBuffer(const ClientBuffer &clientBuffer)
```

Set a client buffer on this tensor memory.

Sets the memory type to Raw and associates the given client buffer.

See also

[QairtTensor_Memory_setClientBuffer](#)

Parameters

clientBuffer – [in] The client buffer to associate with this tensor memory.

Throws

[qairt::Exception](#) – on invalid handle.

```
inline void setClientBuffer(RawBuffer &&buffer)
```

Set a raw memory buffer on this tensor memory.

Constructs a [ClientBuffer](#) from the given [RawBuffer](#) and associates it with this tensor memory. Falls back silently if the backend does not support [ClientBuffer](#) creation.

Parameters

buffer – [in] Raw memory buffer whose data pointer and size are transferred into the new client buffer.

```
inline void setMemHandle(QairtMem_Handle_t memHandle)
```

See also

[QairtTensor_Memory_setMemHandle](#)

```
inline QairtMem_Handle_t getMemHandle() const
```

See also

[QairtTensor_Memory_getMemHandle](#)

```
inline void setRawRetrieveCallbacks(Qairt_GetTensorRawDataFn_t getCallback,  
Qairt_FreeTensorRawDataFn_t freeCallback, void *cookie)
```

See also

[QairtTensor_Memory_setRawRetrieveCallbacks](#)

```
inline void getRawRetrieveCallbacks(Qairt_GetTensorRawDataFn_t *getCallback,  
Qairt_FreeTensorRawDataFn_t *freeCallback, void **cookie) const
```

See also

[QairtTensor_Memory_getRawRetrieveCallbacks](#)

Private Functions

```
inline explicit TensorMemory(const std::shared_ptr<ApiTable> &apiTable)
```

```
inline void prepareToCross() const
```

```
inline void updateAfterCross() const
```

Private Members

```
detail::crossable<detail::non_owning<ClientBuffer>, &interface_type::getClientBuffer,  
&interface_type::setClientBuffer> m_clientBuffer
```

Client buffer associated with this tensor memory when the memory type is Raw.

Friends

```
friend class Api
```

```
class TensorProperties : public qairt::ApiType<TensorProperties,  
QairtTensor_PropertiesV1_t>
```

```
#include <QairtTensor.hpp>
```

Attribute flags describing the role and usage of a tensor within a graph.

Construct directly: TensorProperties(). Attach to a [Tensor](#) via Tensor::setTensorProperties().

Public Functions

```
TensorProperties() noexcept = default
```

```
TensorProperties(const TensorProperties&) = delete
```

```
TensorProperties(TensorProperties&&) noexcept = default
```

```
TensorProperties &operator=(const TensorProperties&) = delete
```

TensorProperties &operator=(**TensorProperties**&&) noexcept = default

inline void setIsInput(bool value)

See also

[QairTensor_Properties_setIsInput](#)

inline void setIsOutput(bool value)

See also

[QairTensor_Properties_setIsOutput](#)

inline void setIsNative(bool value)

See also

[QairTensor_Properties_setIsNative](#)

inline void setIsNull(bool value)

See also

[QairTensor_Properties_setIsNull](#)

inline void setIsStatic(bool value)

See also

[QairTensor_Properties_setIsStatic](#)

inline void setIsOptional(bool value)



[QairTensor_Properties_setIsOptional](#)

inline void setIsUpdatable(bool value)

See also

[QairTensor_Properties_setIsUpdatable](#)

inline bool isInput() const

See also

[QairTensor_Properties_getIsInput](#)

inline bool isOutput() const

See also

[QairTensor_Properties_getIsOutput](#)

inline bool isNative() const

See also

[QairTensor_Properties_getIsNative](#)

inline bool isNull() const

See also

[QairTensor_Properties_getIsNull](#)

inline bool isStatic() const



[QairtTensor_Properties_getIsStatic](#)

inline bool isOptional() const

See also

[QairtTensor_Properties_getIsOptional](#)

inline bool isUpdatable() const

See also

[QairtTensor_Properties_getIsUpdatable](#)

Provide Feedback

Private Functions

inline explicit TensorProperties(const std::shared_ptr<ApiTable> &[apiTable](#))

Friends

***friend class* Api**

**class VectorEncoding : public [qairt::ApiType](#)<[VectorEncoding](#),
[QairtQuantizeParams_VectorEncodingV1_t](#)>**

#include <QairtTensor.hpp>

Vector quantization (VQ) compression parameters, including a codebook descriptor and block-layout configuration.

Construct directly: [VectorEncoding\(const BwAxisScaleOffset& bwAxisScaleOffset, uint32_t rowsPerBlock, uint32_t colsPerBlock, uint32_t vectorStride, uint32_t vectorDimension, uint32_t indexBitwidth\)](#). Attach to a [QuantizeParams](#) via [QuantizeParams::setVectorEncoding\(\)](#).

Public Functions



```
inline ~VectorEncoding()
```

```
VectorEncoding() noexcept = default
```

```
inline VectorEncoding(VectorEncoding &&other) noexcept
```

```
inline VectorEncoding &operator=(VectorEncoding &&other) noexcept
```

```
inline VectorEncoding(const VectorEncoding &other)
```

```
inline VectorEncoding &operator=(const VectorEncoding &other)
```

```
inline VectorEncoding shallowCopy() const
```

```
inline VectorEncoding(const BwAxisScaleOffset &bwAxisScaleOffset, uint32_t  
rowsPerBlock, uint32_t colsPerBlock, uint32_t vectorStride, uint32_t  
vectorDimension, uint32_t indexBitwidth)
```

Construct a vector encoding with the given codebook and block layout.

Parameters

- **bwAxisScaleOffset** – [in] Bit-width per-axis scale-offset encoding for the codebook.
- **rowsPerBlock** – [in] Number of rows per block.
- **colsPerBlock** – [in] Number of columns per block.
- **vectorStride** – [in] Stride between vectors in the compressed representation.

- `indexBitwidth` – [in] Bit-width of codebook indices.

`inline uint32_t getRowsPerBlock() const`

See also

[QairtQuantizeParams_VectorEncoding_getRowsPerBlock](#)

`inline void setRowsPerBlock(uint32_t rowsPerBlock) const`

See also

[QairtQuantizeParams_VectorEncoding_setRowsPerBlock](#)

`inline uint32_t getColsPerBlock() const`

See also

[QairtQuantizeParams_VectorEncoding_getColsPerBlock](#)

`inline void setColsPerBlock(uint32_t colsPerBlock) const`

See also

[QairtQuantizeParams_VectorEncoding_setColsPerBlock](#)

`inline uint32_t getVectorStride() const`

See also

[QairtQuantizeParams_VectorEncoding_getVectorStride](#)

Provide Feedback

```
inline void setVectorStride(uint32_t vectorStride) const
```

See also

[QairtQuantizeParams_VectorEncoding_setVectorStride](#)

```
inline uint32_t getVectorDimension() const
```

See also

[QairtQuantizeParams_VectorEncoding_getVectorDimension](#)

```
inline void setVectorDimension(uint32_t vectorDimension) const
```

See also

[QairtQuantizeParams_VectorEncoding_setVectorDimension](#)

```
inline uint32_t getIndexBw() const
```

See also

[QairtQuantizeParams_VectorEncoding_getIndexBw](#)

```
inline void setIndexBw(uint32_t indexBw) const
```

See also

[QairtQuantizeParams_VectorEncoding_setIndexBw](#)

```
inline const BwAxisScaleOffset &getBwAxisScaleOffset() const
```

Get the bit-width per-axis scale-offset encoding used for the vector codebook.

Throws



Returns

Const reference to the `BwAxisScaleOffset` codebook descriptor.

```
inline void setBwAxisScaleOffset(const BwAxisScaleOffset &bwAxisScaleOffset)
```

Set the bit-width per-axis scale-offset encoding used for the vector codebook.

Parameters

`bwAxisScaleOffset` – [in] Codebook descriptor for the vector quantization.

Throws

`qairt::Exception` – on invalid handle.

```
template<typename T, typename U, typename V>  
inline ApiType(const ApiType<T, U, V> &parent,  
detail::non_owning_handle<handle_type> noh)
```

```
inline explicit ApiType(const std::shared_ptr<T_Table> &apiTable)
```

```
inline ApiType(copy_table_tag_t, const ApiType &other)
```

```
ApiType() noexcept = default
```

```
ApiType(const ApiType&) = delete
```

```
ApiType(ApiType&&) noexcept = default
```

```
ApiType &operator=(const ApiType&) = delete
```

```
ApiType &operator=(ApiType&&) noexcept = default
```

Private Types

```
using CrossableBwAxisScaleOffset =  
detail::crossable<detail::non_owing<BwAxisScaleOffset>,  
&interface_type::getBwAxisScaleOffset, &interface_type::setBwAxisScaleOffset>
```

Private Functions

```
inline void prepareToCross() const
```

```
inline void updateAfterCross() const
```

```
inline explicit VectorEncoding(const std::shared_ptr<ApiTable> &apiTable)
```

Private Members

```
friend Api
```

```
mutable CrossableBwAxisScaleOffset m_bwAxisScaleOffset
```

Bit-width per-axis scale-offset encoding used for the vector codebook.



May contain U.S. and international export controlled information

Light Dark Auto



Provide Feedback

Qualcomm relentlessly innovates to deliver intelligent computing everywhere, helping the world tackle some of its most important challenges. Our leading-edge AI, high performance, low-power computing, and unrivaled connectivity deliver proven solutions that transform major industries. At Qualcomm, we are engineering human progress.



Quick links

- Products
- Support
- Partners
- Contact us
- Developer

Company info

- About us
- Careers
- Investors
- News & media
- Our businesses
- Email Subscriptions

Stay connected

Get the latest Qualcomm and industry information delivered to your inbox.

 [Subscribe](#)

[Manage your subscription](#)

© Qualcomm Technologies, Inc. and/or its affiliated companies.

Snapdragon and Qualcomm branded products are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.

Note: Certain services and materials may require you to accept additional terms and conditions before accessing or using those items.

References to "Qualcomm" may mean Qualcomm Incorporated, or subsidiaries or business units within the Qualcomm corporate structure, as applicable.

Qualcomm Incorporated includes our licensing business, QTL, and the vast majority of our patent portfolio. Qualcomm Technologies, Inc., a subsidiary of Qualcomm Incorporated, operates, along with its subsidiaries, substantially all of our engineering, research and development functions, and substantially all of our products and services businesses, including our QCT semiconductor business.

Materials that are as of a specific date, including but not limited to press releases, presentations, blog posts and webcasts, may have been superseded by subsequent events or disclosures.

Nothing in these materials is an offer to sell or license any of the services or materials referenced herein.